

Informe Proyecto Final: My Letter Locker

Estudiantes:

Brahian Stiven Mosquera Valanta

Jorge Iván Acosta Aristizábal



Universidad del Quindío

Facultad Ingeniería

Ingeniería de Sistemas y Computación

Teoría de Lenguajes Formales

Armenia, Quindío

Mayo de 2026

Índice

1. Introducción teórica	4
1.1. Autómatas finitos	4
1.2. Alfabetos y lenguajes	5
1.3. Relación entre autómatas y expresiones regulares	5
1.4. Diseño de autómatas personalizados	5
Introducción	6
2. Definición y requisitos del proyecto	6
2.1. Requisitos Técnicos	7
2.2. Requisitos Funcionales	7
3. Diseño e implementación	8
3.1. Patrones a identificar	8
3.1.1. Comandos de formato	8
3.2. CommandAutomaton	9
3.3. EmailAutomaton	10
3.4. UrlAutomaton	12
3.4.1. Datos estructurados	13
3.5. Tecnologías y herramientas seleccionadas	13
3.6. Estructura del proyecto	14
3.7. Diseño de interfaz	14
3.8. Algoritmo de cifrado y compresión	16
4. Evidencias	17
4.1. Interfaz Writer	17
4.2. Interfaz Reader	18

4.3. Lectura del mensaje descifrado	19
4.4. Compartición de carta	20
5. Conclusiones	21
5.1. Trabajo futuro	22
6. Bibliografía	23

1. Introducción teórica

La Teoría de Lenguajes Formales proporciona los fundamentos matemáticos para el procesamiento automático de texto y el análisis sintáctico Hopcroft et al., 2006. Esta sección presenta los conceptos formales que sustentan el diseño e implementación de *My Letter Locker*.

1.1. Autómatas finitos

Un autómata finito es un modelo matemático abstracto compuesto por cinco elementos: una tupla $(Q, \Sigma, \delta, q_0, F)$, donde:

- Q es un conjunto finito de estados.
- Σ es un alfabeto finito de símbolos de entrada.
- $\delta : Q \times \Sigma \rightarrow Q$ es la función de transición.
- $q_0 \in Q$ es el estado inicial.
- $F \subseteq Q$ es el conjunto de estados de aceptación.

Según Sipser (2012), un autómata finito determina si una cadena de entrada pertenece a un lenguaje dado, procesando la entrada símbolo a símbolo y modificando su estado interno según las transiciones definidas.

Los autómatas finitos se clasifican en:

- **Deterministas (AFD):** Para cada par (q, a) existe exactamente una transición.
- **No deterministas (AFN):** Pueden existir múltiples transiciones o transiciones con símbolos vacíos (ϵ).

Todo AFN puede convertirse en un AFD equivalente mediante el algoritmo de construcción de subconjuntos Hopcroft et al., 2006.

1.2. Alfabetos y lenguajes

Un **alfabeto** Σ es un conjunto finito no vacío de símbolos Linz, 2012. Un **lenguaje** L sobre un alfabeto Σ es un subconjunto de Σ^* , donde Σ^* denota el conjunto de todas las cadenas (incluyendo la cadena vacía ϵ) formadas por símbolos de Σ .

La operación de concatenación y el operador Kleene permiten construir nuevos lenguajes a partir de existentes:

- $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$
- $L^* = \bigcup_{i=0}^{\infty} L^i$ (clausura de Kleene)

1.3. Relación entre autómatas y expresiones regulares

Un resultado fundamental de la teoría de lenguajes formales establece que los lenguajes reconocidos por autómatas finitos son exactamente aquellos que pueden describirse mediante expresiones regulares Hopcroft et al., 2006. Esta equivalencia permite utilizar autómatas para implementar reconocedores de patrones definidos declarativamente.

Sin embargo, en la implementación práctica de Thompson (1968), las expresiones regulares se compilan internamente en autómatas finitos no deterministas (AFN) y posteriormente en AFD para su ejecución eficiente. Este proceso de compilación es transparente para el usuario, quien define patrones mediante una notación declarativa.

1.4. Diseño de autómatas personalizados

El diseño de un autómata para reconocer un lenguaje específico requiere definir Kelley, 1995:

1. **Alfabeto:** El conjunto de símbolos que el autómata puede procesar.
2. **Estados:** La representación del estado de procesamiento en cada punto.

3. **Transiciones:** Las reglas que determinan el siguiente estado dado el estado actual y el símbolo de entrada.
4. **Estados de aceptación:** Identificación de los estados que representan éxito en el reconocimiento.

La implementación de autómatas en software permite procesamiento en tiempo real, donde cada carácter de entrada se procesa secuencialmente y el resultado se genera incrementalmente Gusfield, 1997.

La Teoría de Lenguajes Formales proporciona los fundamentos matemáticos para el procesamiento automático de texto y el análisis sintáctico. Entre sus herramientas principales se encuentran los autómatas finitos, modelos abstractos compuestos por estados y transiciones que procesan secuencias de entrada símbolo a símbolo. Estos formalismos se aplican tanto al reconocimiento de datos estructurados (correos electrónicos, URLs, fechas) como al análisis de comandos de personalización. El presente proyecto, desarrollado como trabajo final de la asignatura Teoría de Lenguajes Formales de la Universidad del Quindío, aplica estos conceptos al desarrollo de *My Letter Locker*, una aplicación web para compartir cartas digitales de forma privada.

2. Definición y requisitos del proyecto

My Letter Locker es una aplicación web que permite compartir cartas digitales de forma privada, sin necesidad de registro ni almacenamiento en servidor. El contenido de cada carta se cifra mediante un algoritmo de cifrado simétrico y se embebe directamente en la URL, de manera que solo quien posea el enlace y la palabra secreta puede descifrar su contenido.

Para procesar las entradas del usuario, la aplicación emplea autómatas finitos como mecanismo formal de análisis. Estos modelos reconocen tanto datos estructurados (correos electrónicos, URLs, fechas) como comandos de personalización que modifican la apariencia del texto y el entorno visual.

En las siguientes tablas se detallan los requisitos técnicos y funcionales del sistema.

2.1. Requisitos Técnicos

Requisito	Descripción
validación de patrones de datos	Verificación de formato mediante autómatas finitos para campos como correo electrónico, URL, fecha y título
Reconocimiento de comandos	Análisis sintáctico de comandos de formato y carta mediante autómatas finitos
Generación de identificadores	Creación de tokens únicos embebidos en las URLs
Cifrado simétrico	Algoritmo para cifrar y descifrar el contenido de la carta

2.2. Requisitos Funcionales

Requisito	Descripción
Creación sin registro	El usuario redacta el mensaje y define la palabra secreta sin necesidad de cuenta
Cifrado embebido en URL	El contenido se cifra y embebe directamente en la URL, sin almacenamiento en servidor
Comandos de carta	Comandos globales para configurar la palabra secreta, tema visual, tipografía, iluminación y música de fondo
Comandos de formato	Comandos inline para aplicar negrita, cursiva, títulos y otros estilos al texto de la carta
Desbloqueo con palabra secreta	El destinatario debe ingresar la palabra secreta para descifrar el mensaje

3. Diseño e implementación

El desarrollo de *My Letter Locker* se fundamenta en una arquitectura modular que separa claramente los componentes de análisis sintáctico, cifrado, compresión y visualización. Esta sección describe los patrones reconocidos mediante autómatas finitos, las tecnologías utilizadas, la estructura del proyecto y el diseño de la interfaz.

3.1. Patrones a identificar

Como se describió en la Sección 1, la aplicación reconoce patrones mediante autómatas finitos. El diseño de cada autómata sigue los principios definidos en Kelley, 1995, con un alfabeto personalizado y un matcher que coordina su ejecución.

Un alfabeto define los caracteres reconocibles y las transiciones válidas entre estados, proporcionando una separación clara entre la lógica del autómata y los datos que procesa. Por ejemplo:

- `CommandAlphabet` define métodos estáticos para identificar símbolos de inicio/final (\$), caracteres alfabéticos, dígitos y niveles de título.
- `UrlAlphabet` define la validación de esquemas de URL, reconociendo prefijos como `http://`, `https://` y `www.`, además de caracteres válidos en URLs (alfanuméricos, guiones, puntos, dos puntos, barras, etc.).

3.1.1. Comandos de formato

Los comandos siguen una sintaxis declarativa con delimitadores de dólar (\$). Se distinguen dos tipos:

- **Comandos inline:** Se aplican al texto seleccionado (negrita, cursiva).
- **Comandos de línea:** Se colocan al inicio de una línea y aplican formato al contenido siguiente (títulos).

Estructura: `$nombre$valor$` o `$nombre:modificador$valor$`.

Comandos reconocidos:

- `$bold$texto$` — Aplica negrita.
- `$italic$texto$` — Aplica cursiva.
- `$title$ texto` — Encabezado nivel 1.
- `$title:n$ texto` — Encabezado nivel n (2 a 6).

Los autómatas reconocedores implementados en el proyecto son:

3.2. CommandAutomaton

El **CommandAutomaton** reconoce comandos de formato delimitados por el símbolo `$`.

Se define formalmente como la quintupla:

$$M_{cmd} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- $Q = \{Q_0, Q_1, Q_2, Q_3\}$: Conjunto finito de 4 estados.
- $\Sigma = \{\$, a - z, A - Z, 0 - 9, \backslash n\}$: Alfabeto de entrada incluyendo el delimitador de comando, caracteres alfanuméricos y fin de línea.
- $q_0 = Q_0$: Estado inicial donde el autómata busca el inicio de un comando.
- $F = \{Q_0\}$: Conjunto de estados de aceptación.

La función de transición $\delta : Q \times \Sigma \rightarrow Q$ reconoce comandos en dos modalidades:

1. **Comandos inline:** Sintaxis `$nombre$valor$`. El valor se captura entre los delimitadores `$`. Ejemplo: `$bold$texto$` produce texto en negrita.

2. **Comandos de línea:** Sintaxis `$nombre$valor` terminados por `newline` o fin de texto.

Ejemplo: `$title$Encabezado` produce un título.

El flujo de estados es:

- $Q_0 \xrightarrow{\$} Q_1$: Detecta inicio de comando.
- $Q_1 \xrightarrow{\$(comando inline)} Q_2$: Identifica comando inline.
- $Q_1 \xrightarrow{\$(comando línea)} Q_3$: Identifica comando de línea.
- $Q_2 \xrightarrow{\$} Q_0$: Finaliza comando inline al cerrar con \$.
- $Q_3 \xrightarrow{\backslash n} Q_0$: Finaliza comando de línea al encontrar newline.

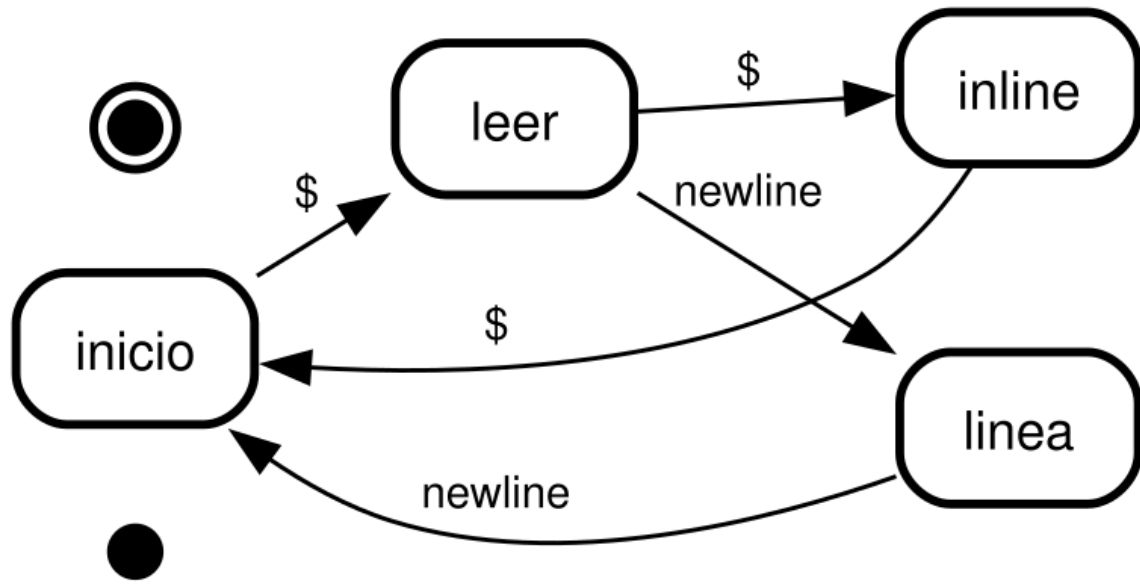


Figura 1: Diagrama de estados del CommandAutomaton. Estados: Q_0 (inicial, buscando inicio de comando), Q_1 (leyendo nombre del comando), Q_2 (capturando valor para comando inline), Q_3 (capturando valor para comando de línea). El autómata distingue entre comandos inline (`$nombre$valor$`) y comandos de línea (`$nombre$valor` seguido de `newline`).

3.3. EmailAutomaton

El **EmailAutomaton** valida direcciones de correo electrónico según el formato `local@dominio.tld`:

$$M_{email} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- $Q = \{Q_0, Q_{local}, Q_{dominio}, Q_{tld}\}$: Estados que representan las fases de reconocimiento del correo.
- Σ : Caracteres alfanuméricos, punto (.), arroba (@), guión bajo (_) y signo más (+).
- $q_0 = Q_0$: Estado inicial.
- $F = \{Q_{dominio}, Q_{tld}\}$: Estados de aceptación.

El reconocimiento sigue la estructura:

1. **Fase local** (Q_{local}): Captura la parte local del correo (antes del @). Acepta caracteres alfanuméricos, punto, guión y plus.
2. **Fase dominio** ($Q_{dominio}$): Captura el dominio (después del @). Reconoce subdominios separados por puntos.
3. **Fase TLD** (Q_{tld}): Captura la extensión del dominio (ej: com, org, edu.co).

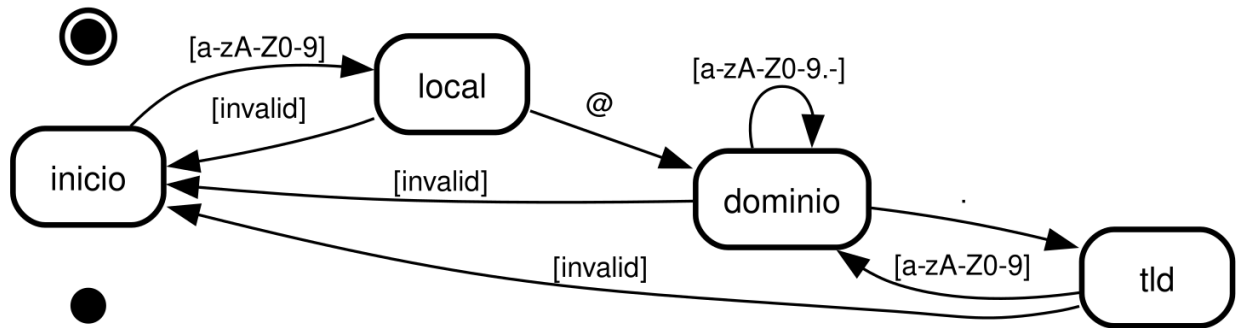


Figura 2: Diagrama de estados del EmailAutomaton. Q_0 espera el inicio de un correo. Q_{local} captura la parte local. $Q_{dominio}$ procesa el dominio después del @. Q_{tld} captura la extensión TLD después del punto final del dominio. El autómata acepta la dirección al detectar un punto en el dominio seguido de caracteres alfanuméricos.

3.4. UrlAutomaton

El **UrlAutomaton** reconoce URLs con esquema `http://`, `https://` o `www.`:

$$M_{url} = (Q, \Sigma, \delta, q_0, F)$$

donde:

- $Q = \{Q_0, Q_{esquema}, Q_{dominio}, Q_{ruta}\}$: Estados para cada fase del reconocimiento de URL.
- Σ : Caracteres válidos en URLs: alfanuméricos, guión, punto, dos puntos, barra, signo de interrogación, numeral, ampersand, igual, porcentaje.
- $q_0 = Q_0$: Estado inicial.
- $F = \{Q_{dominio}, Q_{ruta}\}$: Estados de aceptación.

El reconocimiento procede como:

1. **Detección de esquema** ($Q_{esquema}$): Identifica `http://`, `https://` o `www.`
2. **Captura de dominio** ($Q_{dominio}$): Procesa el nombre de dominio incluyendo subdominios y guiones.
3. **Captura de ruta** (Q_{ruta}): Captura la ruta del recurso después del primer `/`.

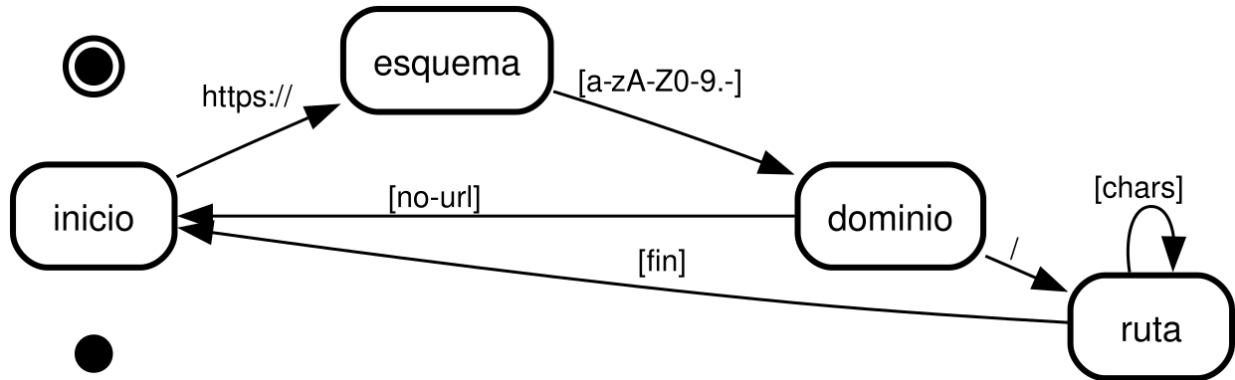


Figura 3: Diagrama de estados del UrlAutomaton. Q_0 detecta el esquema (`http`, `https` o `www`). $Q_{dominio}$ procesa el nombre de dominio. Q_{ruta} captura la ruta del recurso después del primer slash.

Los diagramas de estados fueron generados automáticamente a partir de la especificación formal utilizando la herramienta `state-machine-cat`.

3.4.1. Datos estructurados

Los matchers de datos emplean autómatas específicos para validar:

- **EmailMatcher:** Direcciones de correo electrónico.
- **UrlMatcher:** URLs con protocolo opcional.
- **NewlineMatcher y TabMatcher:** Caracteres especiales de formato de texto.

Estos matchers permiten el resaltado sintáctico en tiempo real dentro del editor, mejorando la experiencia del usuario.

3.5. Tecnologías y herramientas seleccionadas

La pila tecnológica prioriza la simplicidad, el tipado estático y la eliminación de dependencias innecesarias:

Tecnología	Uso en el proyecto
TypeScript	Lenguaje principal con tipado estático
Vite	Bundler y servidor de desarrollo
Tailwind CSS v4	Estilos con utilidades CSS
Navigo	Enrutamiento SPA basado en hash
Handlebars	Motor de plantillas para vistas HTML
CodeMirror 6	Editor de texto enriquecido
Lucide	Iconos SVG para la interfaz
Pako	Compresión gzip para permitir hasta 60 000 caracteres
Web Crypto API	Cifrado AES-GCM y derivación PBKDF2

Se utilizó Web Crypto API nativa del navegador para criptografía, evitando bibliotecas externas. El uso de Tailwind CSS permite un desarrollo rápido de estilos sin archivos CSS adicionales.

3.6. Estructura del proyecto

El código fuente se organiza en módulos funcionales dentro de `src/`:

- `application/` — Punto de entrada, rutas y vistas principales (Writer, Reader).
- `core/` — Motor de autómatas y matchers:
 - `automaton/` — Clase base genérica y tipos compartidos.
 - `matchers/` — Matchers específicos: command, email, url, newline, tab.
 - `analyzer/` — Análisis implícito del contenido.
 - `previewer/` — Previsualización del documento renderizado.
- `encryption/` — Servicio de cifrado y descifrado.
- `utils/` — Utilidades: compresión gzip.
- `history/` — Historial de navegación.

3.7. Diseño de interfaz

La aplicación se divide en dos vistas principales:

- **Writer (/):** Vista de redacción. Incluye editor de texto enriquecido con soporte para comandos, barra de herramientas de formato (negrita, cursiva, títulos), campo para palabra secreta, y opciones de compartición (copiar enlace, WhatsApp, correo electrónico).

La Figura 4 muestra el diseño de la interfaz de escritura.

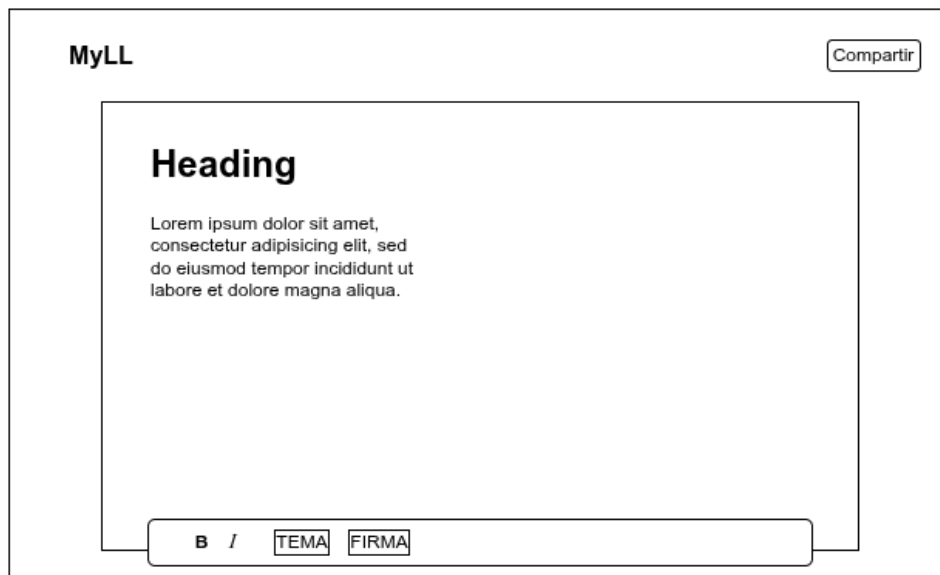


Figura 4: Mockup de la interfaz Writer

- **Reader (/read):** Vista de lectura. Solicita la palabra secreta al destinatario, descifra el contenido y lo renderiza con formato HTML. Si el cifrado está deshabilitado, permite lectura directa del enlace.

La Figura 5 muestra el diseño de la interfaz de lectura.

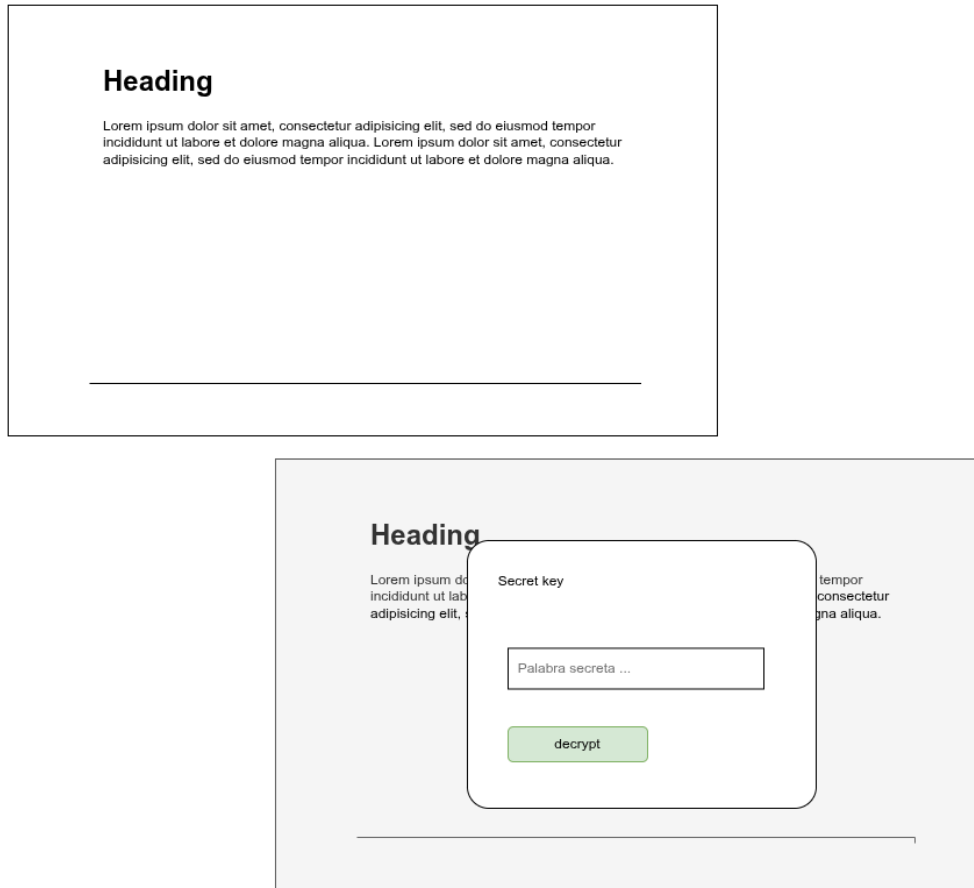


Figura 5: Mockup de la interfaz Reader

La navegación entre vistas se gestiona mediante Navigo, manteniendo la arquitectura de aplicación de página única (SPA). El sistema soporta modo claro y oscuro, persistido en el almacenamiento local del navegador.

3.8. Algoritmo de cifrado y compresión

El flujo de procesamiento combina compresión y cifrado para permitir cartas de hasta 60 000 caracteres:

1. **Compresión:** El texto plano se comprime mediante gzip (biblioteca Pako), reduciendo significativamente el tamaño antes del cifrado.

2. **Derivación de clave:** PBKDF2 con 100 000 iteraciones y SHA-256 deriva una clave AES-256 a partir de la palabra secreta.
3. **Cifrado:** AES-GCM con vector de inicialización aleatorio proporciona cifrado autenticado.
4. **Formato de salida:** Base64url de [salt (16 bytes) | iv (12 bytes) | ciphertext].

El flujo inverso (descifrado) invierte estos pasos: descifrar, descomprimir y procesar los comandos de formato mediante los autómatas reconocedores.

4. Evidencias

En esta sección se presentan las capturas de pantalla y pruebas funcionales que demuestran la correcta implementación del sistema.

4.1. Interfaz Writer

La Figura 6 muestra la interfaz de redacción con el editor de texto enriquecido, la barra de herramientas y las opciones de compartición.

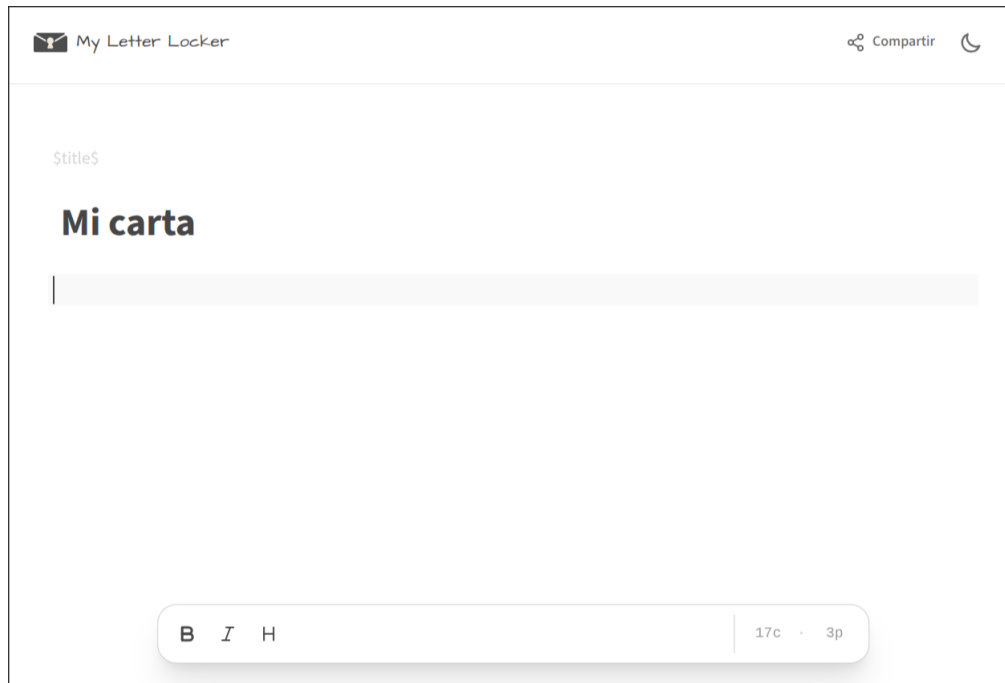


Figura 6: Captura de la interfaz Writer

4.2. Interfaz Reader

La Figura 7 presenta la pantalla de desbloqueo donde el destinatario ingresa la palabra secreta.

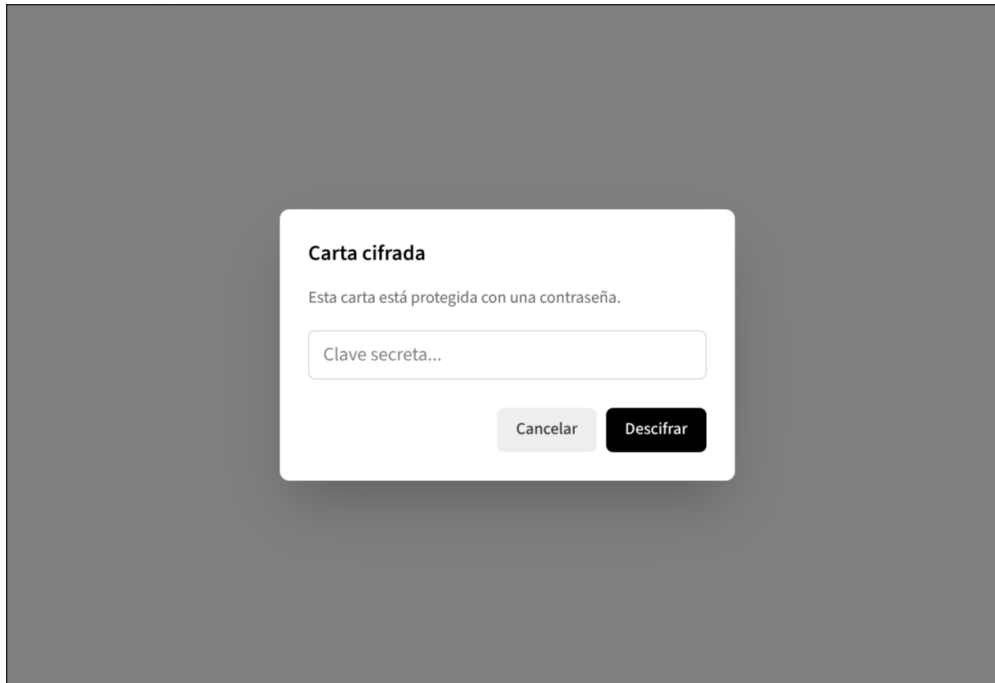


Figura 7: Captura de la interfaz Reader pidiendo palabra secreta

4.3. Lectura del mensaje descifrado

La Figura 8 muestra el contenido descifrado y renderizado con los comandos de formato aplicados.

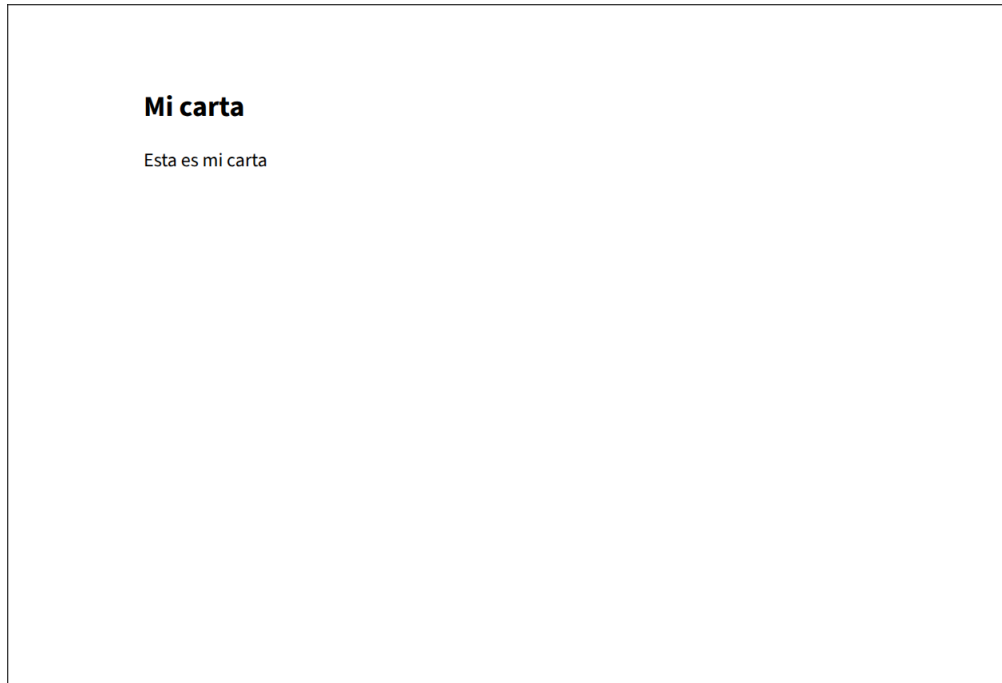


Figura 8: Mensaje descifrado con formato

4.4. Compartición de carta

La Figura 9 evidencia la generación del enlace con contenido cifrado embebido y las opciones de compartición disponibles.

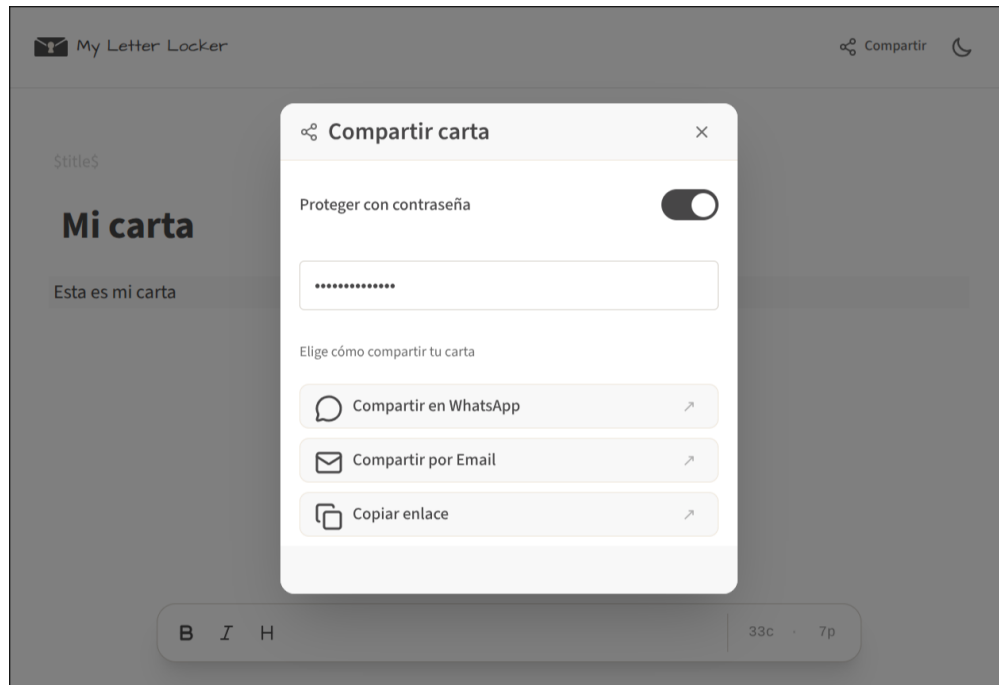


Figura 9: Ventana de compartición con enlace cifrado

5. Conclusiones

El desarrollo de *My Letter Locker* demuestra la aplicación práctica de los conceptos de teoría de lenguajes formales en un sistema real. A través de autómatas finitos personalizados, la aplicación logra reconocer y procesar comandos de formato y datos estructurados sin necesidad de expresiones regulares externas.

Los principales logros del proyecto incluyen:

- **Arquitectura modular:** La separación clara entre autómatas, matchers y servicios permite un código mantenible y extensible. Cada componente tiene responsabilidad específica.
- **Cifrado autenticado:** El uso de AES-GCM con PBKDF2 garantiza tanto la confidencialidad como la integridad de los mensajes, protegiendo contra manipulación no autorizada.

- **Compresión eficiente:** La integración de gzip mediante la biblioteca Pako permite compartir mensajes de hasta 60 000 caracteres mediante URLs, superando las limitaciones habituales de longitud de enlaces.
- **Sin servidor:** El diseño elimina la necesidad de almacenamiento permanente, reduciendo costos y complejidad operacional. Toda la información se embebe en la URL.
- **Sintaxis declarativa:** Los comandos de formato inspirados en LaTeX proporcionan una notación familiar para redactores técnicos, con una gramática simple pero expresiva.

El proyecto evidencia cómo los fundamentos teóricos de autómatas y lenguajes formales permiten construir soluciones elegantes a problemas prácticos de procesamiento de texto.

5.1. Trabajo futuro

Como líneas de mejora para futuras iteraciones se identifican:

- Soporte para adjuntos e imágenes embebidas mediante codificación Base64.
- Historial de versiones con diferenciación criptográfica.
- Integración con servicios de almacenamiento distribuido (IPFS, Arweave).
- Comandos adicionales: listas, citas, código fuente con sintaxis coloreada.
- Internacionalización y temas visuales personalizables.

6. Bibliografía

Referencias

- Gusfield, D. (1997). *Algorithms on Strings, Trees, and Sequences* [Cover treatment of string matching and automata theory]. Cambridge University Press.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3.^a ed.) [Referencia clásica sobre teoría de autómatas y lenguajes formales]. Pearson Addison-Wesley.
- Kelley, D. (1995). *Automata and Formal Languages: An Introduction* [Introducción a autómatas y lenguajes formales con énfasis en aplicaciones prácticas]. Prentice Hall.
- Linz, P. (2012). *An Introduction to Formal Languages and Automata* (5.^a ed.) [Introducción accesible a lenguajes formales y autómatas finitos]. Jones; Bartlett Learning.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3.^a ed.) [Texto estándar para el estudio de autómatas finitos y complejidad computacional]. Cengage Learning.
- Thompson, K. (1968). Programming Techniques: Regular Expression Search Algorithm [Artículo original que describe la implementación de expresiones regulares en Unix]. *Communications of the ACM*, 11(6), 419-422.